

-  GUÍA DE INTEGRACIÓN — BRAYLIN
 -  ARCHIVOS QUE SON TU RESPONSABILIDAD
 -  PRE-REQUISITOS
 -  PASO A PASO COMPLETO
 - PASO 1 — Clonar el repositorio (solo si no lo tienes)
 - PASO 2 — Configurar tu identidad Git
 - PASO 3 — Asegurarte de estar en develop actualizada
 - PASO 4 — Crear tu rama de trabajo
 - PASO 5 — Compilar para verificar que funciona
 - PASO 6 — Agregar SOLO tu carpeta al staging
 - PASO 7 — Ver exactamente qué cambios vas a subir
 - PASO 8 — Hacer commit
 - PASO 9 — Subir tu rama a GitHub
 - PASO 10 — Verificar en GitHub
 - PASO 11 — Crear el Pull Request
 -  SI NECESITAS ACTUALIZAR TU RAMA CON CAMBIOS DE DEVELOP
 -  PROBLEMAS COMUNES Y SOLUCIONES
 - El JAR no compiló
 - Error "src refsPEC feature/braylin-event-service does not match any"
 - "Updates were rejected because the remote contains work"
 - Accidentalmente modifiqué archivos de otro servicio
 -  CHECKLIST PERSONAL — BRAYLIN
 - Antes de crear la rama
 - Antes del commit
 - Antes de crear el PR
 - Al crear el PR
 -  INFORMACIÓN TÉCNICA DE TU SERVICIO



GUÍA DE INTEGRACIÓN — BRAYLIN

Componentes a cargo: `event-service/` **Rama de trabajo:** `feature/braylin-event-service` **Ejecutar en:** FASE 4 (después de que Rachel y Andrea hayan mergeado)

ARCHIVOS QUE SON TU RESPONSABILIDAD

```
event-service/
├── src/
│   ├── main/
│   │   ├── java/com/eventos/event_service/
│   │   │   ├── EventServiceApplication.java
│   │   │   ├── DataInitializer.java
│   │   │   ├── controller/
│   │   │   │   ├── CommentController.java ← NUEVO
│   │   │   │   ├── EventController.java
│   │   │   │   └── HealthController.java
│   │   │   ├── dto/
│   │   │   │   ├── CommentRequest.java ← NUEVO
│   │   │   │   ├── CommentResponse.java ← NUEVO
│   │   │   │   ├── EventMapper.java
│   │   │   │   ├── EventRequest.java
│   │   │   │   └── EventResponse.java
│   │   │   ├── entity/
│   │   │   │   ├── Comment.java ← NUEVO
│   │   │   │   └── Event.java
│   │   │   ├── exception/
│   │   │   │   ├── EventNotFoundException.java
│   │   │   │   └── GlobalExceptionHandler.java
│   │   │   ├── repository/
│   │   │   │   ├── CommentRepository.java ← NUEVO
│   │   │   │   └── EventRepository.java
│   │   │   └── service/
│   │   │       ├── CommentService.java ← NUEVO
│   │   │       ├── EventService.java
│   │   │       └── EventServiceImpl.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
├── .mvn/
├── pom.xml
└── mvnw / mvnw.cmd
```

 Los archivos marcados con ← **NUEVO** son archivos que agregaste recientemente (feature de comentarios). Asegúrate de incluirlos todos.

PRE-REQUISITOS

```
# Verificar Git instalado
git --version
# Resultado esperado: git version 2.x.x

# Verificar Java 17
java -version
# Resultado esperado: openjdk version "17.x.x"

# Verificar Maven Wrapper en event-service
ls event-service/mvnw
# Si no existe, usa: mvn (Maven global)
```



PASO A PASO COMPLETO

PASO 1 — Clonar el repositorio (solo si no lo tienes)

```
git clone [URL_DEL_REPO]
cd Eventos-Microservicios
```

PASO 2 — Configurar tu identidad Git

```
git config user.name "Braylin"
git config user.email "tu-email@gmail.com"
```

PASO 3 — Asegurarte de estar en develop actualizada

⚠ IMPORTANTE: Ejecuta este paso SOLO cuando Rachel te avise que las fases 1, 2 y 3 ya están mergeadas en **develop**.

```
git checkout develop
git pull origin develop
```

Verifica que ves los commits de Rachel y Andrea:

```
git log --oneline -10
# Debes ver commits de auth-service, notification-service, user-service, gateway
```

PASO 4 — Crear tu rama de trabajo

```
git checkout -b feature/braylin-event-service
```

Confirma:

```
git branch
# Debe mostrar: * feature/braylin-event-service
```

PASO 5 — Compilar para verificar que funciona

```
cd event-service
./mvnw package -DskipTests
# En Windows sin bash: mvnw.cmd package -DskipTests
```

Si compila exitosamente verás:

```
[INFO] BUILD SUCCESS
[INFO] Total time: X:XX min
```

Volver a la raíz:

```
cd ..
```

PASO 6 — Agregar SOLO tu carpeta al staging

⚠ **REGLA CRÍTICA:** Solamente agrega `event-service/`. Nada más.

```
# Agregar toda la carpeta event-service
git add event-service/

# VERIFICAR inmediatamente
git status
```

Salida esperada:

```
On branch feature/braylin-event-service
Changes to be committed:
  modified:   event-
service/src/main/java/com/eventos/event_service/controller/CommentController.java
  modified:   event-
service/src/main/java/com/eventos/event_service/controller/EventController.java
  modified:   event-
service/src/main/java/com/eventos/event_service/dto/CommentRequest.java
  modified:   event-
service/src/main/java/com/eventos/event_service/dto/CommentResponse.java
  modified:   event-
service/src/main/java/com/eventos/event_service/entity/Comment.java
  modified:   event-
service/src/main/java/com/eventos/event_service/repository/CommentRepository.java
  modified:   event-
service/src/main/java/com/eventos/event_service/service/CommentService.java
  modified:   event-
service/src/main/java/com/eventos/event_service/service/EventService.java
  modified:   event-
service/src/main/java/com/eventos/event_service/service/EventServiceImpl.java
  modified:   event-service/src/main/resources/application.properties
```

✅ Si solo aparecen archivos de `event-service/`, estás bien. ❌ Si aparece algo de otras carpetas, quítalo:

```
git restore --staged auth-service/
git restore --staged user-service/
git restore --staged gateway/
git restore --staged inscrip-service/
git restore --staged frontend-v2/
```

PASO 7 — Ver exactamente qué cambios vas a subir

```
# Ver diferencias de archivos específicos
git diff --staged event-
service/src/main/java/com/eventos/event_service/service/EventServiceImpl.java
```

PASO 8 — Hacer commit

```
git commit -m "feat(event): event-service completo con sistema de comentarios

- CRUD completo de eventos (EventController, EventService, EventServiceImpl)
- Sistema de comentarios: CommentController, CommentService, CommentRepository
- Entidades: Event, Comment con relación JPA
- DataInitializer para datos de prueba
- application.properties configurado para Railway MySQL (event_db)"
```

PASO 9 — Subir tu rama a GitHub

Primera vez:

```
git push -u origin feature/braylin-event-service
```

Cambios adicionales:

```
git push origin feature/braylin-event-service
```

Si necesitas sobrescribir (force push seguro):

```
git push origin feature/braylin-event-service --force-with-lease
```

¿Cuándo necesitarías force push?

- Corregiste el mensaje de un commit (`git commit --amend`)
- Reorganizaste commits (`git rebase`)
- Subiste algo incorrecto y necesitas reemplazarlo

PASO 10 — Verificar en GitHub

1. Abre el repositorio en GitHub
2. Selecciona la rama `feature/braylin-event-service`
3. Verifica que el árbol de archivos solo muestra cambios en `event-service/`
4. Revisa que están todos los archivos nuevos (CommentController, Comment, etc.)

PASO 11 — Crear el Pull Request

Por GitHub UI:

1. GitHub te mostrará un banner: **"Compare & pull request"** → clic
2. Configura:
 - **Base:** `develop` ← ¡muy importante!
 - **Compare:** `feature/braylin-event-service`
 - **Título:** `feat(event): event-service con CRUD de eventos y comentarios`
 - **Descripción:**

```
## Cambios en event-service

### Funcionalidades
- CRUD completo de eventos (crear, leer, actualizar, eliminar)
- Sistema de comentarios en eventos (nuevo)
- Datos iniciales de prueba (DataInitializer)

### Archivos nuevos
- CommentController.java
- CommentService.java
- CommentRepository.java
- Comment.java (entidad)
- CommentRequest.java / CommentResponse.java (DTOs)

### Archivos modificados
- EventController.java
- EventService.java / EventServiceImpl.java
- application.properties

### Pruebas realizadas
- [ ] Compila sin errores
- [ ] GET /events retorna lista
- [ ] POST /events crea evento
- [ ] POST /events/{id}/comments agrega comentario
```

3. Clic "Create Pull Request"

Por CLI:

```
gh pr create \  
  --base develop \  
  --head feature/braylin-event-service \  
  --title "feat(event): event-service con CRUD de eventos y comentarios" \  
  --body "CRUD completo de eventos + sistema de comentarios. Incluye entidades JPA, repositorios, servicios y controladores."
```

SI NECESITAS ACTUALIZAR TU RAMA CON CAMBIOS DE DEVELOP

Si entre que creaste tu rama y que haces el PR, alguien mergeó cambios a **develop**:

```
# 1. Ir a develop y actualizar  
git checkout develop  
git pull origin develop  
  
# 2. Volver a tu rama  
git checkout feature/braylin-event-service  
  
# 3. Traer los cambios de develop a tu rama  
git merge develop  
  
# 4. Si hay conflictos (no debería haber porque no compartes carpetas)  
# Git los marcará en los archivos - abre el archivo y elige qué versión conservar  
# Luego: git add [archivo-resuelto] && git commit  
  
# 5. Push actualizado  
git push origin feature/braylin-event-service
```

PROBLEMAS COMUNES Y SOLUCIONES

El JAR no compiló


```
cd event-service

# Ver el error completo
./mvnw package -DskipTests 2>&1 | tail -50

# Si el error es de dependencias
./mvnw dependency:resolve

# Si es un error de Java version
java -version # Debe ser 17+
```

Error "src refsPEC feature/braylin-event-service does not match any"

```
# Verificar que hiciste al menos un commit
git log --oneline -5

# Si no hay commits, agrega y commitea primero
git add event-service/
git commit -m "feat(event): event-service inicial"
git push -u origin feature/braylin-event-service
```

"Updates were rejected because the remote contains work"

```
# Opción 1: Pull primero (si alguien más también trabajó en esta rama)
git pull origin feature/braylin-event-service

# Opción 2: Force push (si eres el único en esta rama)
git push origin feature/braylin-event-service --force-with-lease
```

Accidentalmente modifiqué archivos de otro servicio

```
# Ver qué está en staging
git status

# Quitar del staging (el archivo queda modificado localmente pero no se sube)
```

```
git restore --staged auth-service/  
git restore --staged user-service/
```

```
# Si quieres DESHACER los cambios locales también (¡cuidado! esto borra tus  
cambios)  
git checkout -- auth-service/
```



CHECKLIST PERSONAL — BRAYLIN

Antes de crear la rama

- ☐ `git checkout develop && git pull origin develop` ejecutado
- ☐ Confirmé con Rachel que las Fases 1, 2 y 3 están mergeadas

Antes del commit

- ☐ `event-service/` compila sin errores (`./mvnw package -DskipTests`)
- ☐ Solo `event-service/` está en staging (`git status` verificado)
- ☐ Todos los archivos nuevos están incluidos (`Comment.java`, `CommentController.java`, etc.)
- ☐ Mensaje de commit es descriptivo

Antes de crear el PR

- ☐ `git push origin feature/braylin-event-service` exitoso
- ☐ Verificado en GitHub que solo aparecen cambios de `event-service/`

Al crear el PR

- ☐ **Base:** `develop` (NO `main`)
 - ☐ **Compare:** `feature/braylin-event-service`
 - ☐ Descripción del PR completada
 - ☐ Notifiqué a Rachel que el PR está listo para revisión
-

INFORMACIÓN TÉCNICA DE TU SERVICIO

event-service:

- Puerto: 8083
- Base de datos: event_db (Railway MySQL)
- Ruta en el Gateway: /events/** → http://localhost:8083

Endpoints principales:

GET	/events	→ Listar todos los eventos
GET	/events/{id}	→ Obtener evento por ID
POST	/events	→ Crear evento
PUT	/events/{id}	→ Actualizar evento
DELETE	/events/{id}	→ Eliminar evento
GET	/events/{id}/comments	→ Comentarios del evento (NUEVO)
POST	/events/{id}/comments	→ Agregar comentario (NUEVO)
GET	/events/health	→ Health check

Dependencias de tu servicio:

- No depende directamente de otros microservicios
- El frontend y inscrip-service consumen tus endpoints